HOA SEN UNIVERSITY

www.hoasen.edu.vn

LARAVEL 4

MIGRATIONS & SEEDING & QUERY BUILDER

Contents

- Migrations
- Seeding
- Query builder

Introduction

- MySQL 5.6+ (Version Policy)
- PostgreSQL 9.4+ (Version Policy)
- SQLite 3.8.8+
- SQL Server 2017+ (Version Policy)

Configuration

```
'connections' => [

    'sqlite' => [
        'driver' => 'sqlite',
        'database' => env('DB_DATABASE', database_path('database.sqlite')),
        'prefix' => '',
    ],

    'mysql' => [
        'driver' => 'mysql',
        'host' => env('DB_HOST', '127.0.0.1'),
        'port' => env('DB_PORT', '3306'),
        'database' => env('DB_DATABASE', 'forge'),
        'username' => env('DB_USERNAME', 'forge'),
        'password' => env('DB_PASSWORD', ''),
        'unix_socket' => env('DB_SOCKET', ''),
        'charset' => 'utf8',
        'collation' => 'utf8_unicode_ci',
        'prefix' => '',
        'strict' => false,
        'engine' => null,
    ],

    'pgsql' => [
        'driver' => 'pgsql',
        'host' => env('DB_HOST', '127.0.0.1'),
        'port' => env('DB_PORT', '5432'),
        'database' => env('DB_DATABASE', 'forge'),
        'username' => env('DB_USERNAME', 'forge'),
        'password' => env('DB_PASSWORD', ''),
        'charset' => 'utf8',
        'prefix' => '',
        'schema' => 'public',
        'sslmode' => 'prefer',
```

The configuration for database access lives in config/database.php and .env. Like many other configuration areas in Laravel, you can define multiple “connections” and then decide which the code will use by default.

```
$users = DB::connection('secondary')->select('select * from users');
```

MIGRATIONS

- Generating Migrations
- Running Migrations
- Tables
- Columns

Generating Migrations

Migrations are like version control for your database, allowing your team to modify and share the application's database schema. Migrations are typically paired with Laravel's schema builder to build your application's database schema.

```
php artisan make:migration create_users_table
```

```
php artisan make:migration create_users_table --create=users

php artisan make:migration add_votes_to_users_table --table=users
```

The --table and --create options may also be used to indicate the name of the table

HOA SEN UNIVERSITY

Migration Structure

- The up method is used to add new tables, columns, or indexes to your database.
- The down method should reverse the operations performed by the up method.

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
class CreateUsersTable extends Migration
{
    public function up()
    {
      Schema::create('users', function (Blueprint $table) {
          $table->id();
          $table->string('name');
          $table->string('email')->unique();
          $table->string('role');
          $table->timestamp('email_verified_at')->nullable();
          $table->string('password');
          $table->rememberToken();
          $table->timestamps();
        );
    }
    public function down()
    {
            Schema::dropIfExists('users');
    }
}
```

Running Migrations

```
php artisan migrate
```

```
php artisan migrate --force
```

```
php artisan migrate:rollback
```

```
php artisan migrate:reset
```

Running Migrations

- The migrate:refresh command will roll back all of your migrations and then execute the migrate command. This command effectively re-creates your entire database:

```
php artisan migrate:refresh
```

```
php artisan migrate:refresh --step=5
```

Tables

- Checking For Table / Column Existence

```
if (Schema::hasTable('users')) {
    //
}

if (Schema::hasColumn('users', 'email')) {
    //
}
```

```
Schema::connection('foo')->create('users',
function (Blueprint $table) {
    $table->id();
});
```

- Renaming / Dropping Tables

```
Schema::rename($from, $to);

Schema::drop('users');
Schema::dropIfExists('users');
```

Command	Description
`$table->id();`	Alias of `$table->bigIncrements('id')`.
`$table->foreignId('user_id');`	Alias of `$table->unsignedBigInteger('user_id')`.
`$table->bigIncrements('id');`	Auto-incrementing UNSIGNED BIGINT (primary key) equivalent column.
`$table->bigInteger('votes');`	BIGINT equivalent column.
`$table->binary('data');`	BLOB equivalent column.
`$table->boolean('confirmed');`	BOOLEAN equivalent column.
`$table->char('name', 100);`	CHAR equivalent column with a length.
`$table->date('created_at');`	DATE equivalent column.
`$table->dateTime('created_at', 0);`	DATETIME equivalent column with precision (total digits).
`$table->decimal('amount', 8, 2);`	DECIMAL equivalent column with precision (total digits) and scale (decimal digits).
`$table->double('amount', 8, 2);`	DOUBLE equivalent column with precision (total digits) and scale (decimal digits).
`$table->enum('level', ['easy', 'hard']);`	ENUM equivalent column.
`$table->float('amount', 8, 2);`	FLOAT equivalent column with a precision (total digits) and scale (decimal digits).
`$table->increments('id');`	Auto-incrementing UNSIGNED INTEGER (primary key) equivalent column.
`$table->integer('votes');`	INTEGER equivalent column.
`$table->ipAddress('visitor');`	IP address equivalent column.
`$table->json('options');`	JSON equivalent column.

Column Modifiers

The following list contains all available column modifiers. This list does not include the index modifiers:

Modifier	Description
->after('column')	Place the column "after" another column (MySQL)
->autoIncrement()	Set INTEGER columns as auto-increment (primary key)
->charset('utf8mb4')	Specify a character set for the column (MySQL)
->comment('my comment')	Add a comment to a column (MySQL/PostgreSQL)
->default($value)	Specify a "default" value for the column
->first()	Place the column "first" in the table (MySQL)
->nullable($value = true)	Allows (by default) NULL values to be inserted into the column
->storedAs($expression)	Create a stored generated column (MySQL)
->unsigned()	Set INTEGER columns as UNSIGNED (MySQL)
->useCurrent()	Set TIMESTAMP columns to use CURRENT_TIMESTAMP as default value
->always()	Defines the precedence of sequence values over input for an identity column (PostgreSQL)

Modifying Columns (1)

- Updating Column Attributes

  The change method allows you to modify type and attributes of existing columns

```
Schema::table('users', function (Blueprint $table) {
    $table->string('name', 50)->change();
});
```

```
Schema::table('users', function (Blueprint $table) {
    $table->string('name', 50)->nullable()->change();
});
```

Modifying Columns (2)

- Renaming Columns

```
Schema::table('users', function (Blueprint $table) {
    $table->renameColumn('from', 'to');
});
```

- Dropping Columns

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn('votes');
});
```

```
Schema::table('users', function (Blueprint $table) {
    $table->dropColumn(['votes', 'avatar', 'location']);
});
```

SEEDING

- Writing Seeders
- Running Seeders

Writing Seeders

- Laravel includes a simple method of seeding your database with test data using seed classes. All seed classes are stored in the database/seeds directory
- To generate a seeder, execute the make:seeder Artisan command

```
php artisan make:seeder UserSeeder
```

```
<?php
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;
class DatabaseSeeder extends Seeder {
    public function run() {
        DB::table('users')->insert([
            'name' => Str::random(10),
            'email' =>
            Str::random(10).'@gmail.com',
            'password' =>
            Hash::make('password'),
        ]);
    }
}
```

Using Model Factories

- You can use model factories to conveniently generate large amounts of database records
  - For example, let's create 50 users and attach a relationship to each user:

```
public function run()
{
    factory(App\User::class, 50)->create()->each(function ($user) {
        $user->posts()->save(factory(App\Post::class)->make());
    });
}
```

Running Seeders

```php
public function run()
{
    $this->call([
        UserSeeder::class,
        PostSeeder::class,
        CommentSeeder::class,
    ]);
}
```

```
php artisan db:seed

php artisan db:seed --class=UserSeeder

php artisan db:seed --force
```

QUERY BUILDER

- Retrieving Results
- Selects
- Raw Expressions
- Joins
- Where clauses
- Inserts
- Updates
- Deletes

Retrieving Results

- The Laravel query builder uses PDO parameter binding to protect your application against SQL injection attacks. There is no need to clean strings being passed as bindings.

```
namespace App\Http\Controllers;
use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\DB;

class UserController extends Controller {
    public function index() {
        $users = DB::table('users')->get();
        return view('user.index', ['users' =>
        $users]);
    }
}
```

Retrieving A Single Row / Column From A Table

```
$user = DB::table('users')->where('name', 'John')->first();
echo $user->name;
```

```
$email = DB::table('users')->where('name', 'John')->value('email');
```

To retrieve a single row by its id column value, use the find method:

```
$user = DB::table('users')->find(3);
```

Retrieving A List Of Column Values

A custom key column

```
$roles = DB::table('roles')->pluck('title', 'name');
foreach ($roles as $name => $title) {
        echo $title;
}
```

Chunking Results

- This method retrieves a small chunk of the results at a time and feeds each chunk into a Closure for processing. You may stop further chunks from being processed by returning false from the Closure

```
DB::table('users')->orderBy('id')->chunk(100, function ($users)
{
        // Process the records...
        return false; //stop processing
});
```

Aggregates

- The query builder also provides a variety of aggregate methods such as count, max, min, avg, and sum

```
$users = DB::table('users')->count();
$price = DB::table('orders')->max('price');
```

```
$price = DB::table('orders')
                ->where('finalized', 1)
                ->avg('price');
```

```
return DB::table('orders')->where('finalized', 1)->exists();

return DB::table('orders')->where('finalized', 1)->doesntExist();
```

Selects

- Using the select method, you can specify a custom select clause for the query

```
$users = DB::table('users')->select('name', 'email
as user_email')->get();
```

```
$users = DB::table('users')->distinct()->get();
```

- If you already have a query builder instance and you wish to add a column to its existing select clause, you may use the addSelect method:

```
$query = DB::table('users')->select('name');

$users = $query->addSelect('age')->get();
```

Raw Expressions

```
$users = DB::table('users')
->select(DB::raw('count(*) as user_count, status'))
                    ->where('status', '<>', 1)
                    ->groupBy('status')
                    ->get();
```

- Raw statements will be injected into the query as strings, so you should be extremely careful to not create SQL injection vulnerabilities.

Raw Methods (1)

Instead of using DB::raw, you may also use the following methods to insert a raw expression into various parts of your query

- selectRaw

  The selectRaw method can be used in place of addSelect(DB::raw(...)). This method accepts an optional array of bindings as its second argument

```
$orders = DB::table('orders')
          ->selectRaw('price * ? as
          price with tax', [1.0825])
```

- whereRaw / orWhereRaw

  The whereRaw and orWhereRaw methods can be used to inject a raw where clause into your query.

```
$orders = DB::table('orders')
          ->whereRaw('price > IF(state = "TX", ?,
          100)', [200])
          ->get();
```

Raw Methods (2)

- havingRaw / orHavingRaw

  The havingRaw and orHavingRaw methods may be used to set a raw string as the value of the having clause

```
$orders = DB::table('orders')
          ->select('department', DB::raw('SUM(price)
          as total_sales'))
          ->groupBy('department')
          ->havingRaw('SUM(price) > ?', [2500])
          ->get();
```

- orderByRaw

  The orderByRaw method may be used to set a raw string as the value of the order by clause:

```
$orders = DB::table('orders')
          ->orderByRaw('updated_at - created_at
          DESC')
          ->get();
```

Raw Methods (3)

- groupByRaw

  The groupByRaw method may be used to set a raw string as the value of the group by clause:

```
$orders = DB::table('orders')
          ->select('city', 'state')
          ->groupByRaw('city, state')
          ->get();
```

Joins

- Inner Join Clause

```
$users = DB::table('users')
          ->join('contacts', 'users.id', '=',
          'contacts.user_id')
          ->join('orders', 'users.id', '=',
          'orders.user_id')
          ->select('users.*', 'contacts.phone',
          'orders.price')
          ->get();
```

- Left Join / Right Join Clause

```
$users = DB::table('users')
          -> leftJoin  ('posts', 'users.id', '=',
          'posts.user_id')
          ->get();
```

rightJoin

Where Clauses

```
$users = DB::table('users')
    ->where('votes', 100)->get();
    //equal
$users = DB::table('users')
            ->where('votes', '>=',
            100)->get();
$users = DB::table('users')
            ->where('votes', '<>',
            100)->get();
$users = DB::table('users')
            ->where('name', 'like',
            'T%')->get();
```

```
$users = DB::table('users')->where([
                ['status', '=', '1'],
                ['subscribed', '<>',
                '1'],
            ])->get();
```

```
$users = DB::table('users')
        ->where('votes', '>', 100)
        ->orWhere('name', 'John')
        ->get();
```

```
$users = DB::table('users')
            ->where('votes', '>', 100)
            ->orWhere(function($query) {
                        $query->where('name', 'Abigail')
                            ->where('votes', '>', 50);
            })->get();
//SQL: select * from users where votes > 100 or (name = 'Abigail' and votes > 50)
```

Where Clauses

- whereBetween / orWhereBetween
- whereNotBetween / orWhereNotBetween
- whereIn / whereNotIn / orWhereIn / orWhereNotIn
- whereNull / whereNotNull / orWhereNull / orWhereNotNull
- whereColumn / orWhereColumn

```
$users = DB::table('users')
          ->whereColumn('first_name', 'last_name')
          ->get();
```

```
$users = DB::table('users')
    ->whereColumn('updated_at','>','created_at')
    ->get();
```

- ...

Inserts

```
DB::table('users')->insert([
            ['email' => 'taylor@example.com', 'votes' => 0],
            ['email' => 'dayle@example.com', 'votes' => 0],
]);
```

The insertOrIgnore method will ignore duplicate record errors while inserting records into the database:

```
DB::table('users')->insertOrIgnore([
            ['id' => 1, 'email' => 'taylor@example.com'],
            ['id' => 2, 'email' => 'dayle@example.com'],
]);
```

If the table has an auto-incrementing id, use the insertGetId method to insert a record and then retrieve the ID:

```
$id = DB::table('users')->insertGetId(
            ['email' => 'john@example.com', 'votes' => 0]
);
```

Updates

```
$affected = DB::table('users')
              ->where('id', 1)
              ->update(['votes' => 1]);
```

- The updateOrInsert method will first attempt to locate a matching database record using the first argument's column and value pairs. If the record exists, it will be updated with the values in the second argument. If the record can not be found, a new record will be inserted with the merged attributes of both arguments

```
DB::table('users')
    ->updateOrInsert(
    ['email' => 'john@example.com','name' => 'John'],
    ['votes' => '2'] );
```

Deletes

```
DB::table('users')->delete();

DB::table('users')->where('votes', '>', 100)->delete();
```

Q&A